

Section A – Conceptual Understanding

1. Explain why REST APIs are preferred for mobile and frontend-heavy apps.

REST APIs are preferred for mobile and frontend-heavy applications because they provide a lightweight, flexible, and scalable way for applications to communicate with servers. REST APIs mainly use JSON format, which is smaller in size and faster to transfer compared to XML, making it ideal for mobile devices and web applications where performance and bandwidth are important.

REST APIs are platform-independent, meaning frontend applications built with React, Angular, Vue, Flutter, or mobile apps can easily connect with the backend without depending on a specific programming language. This separation between frontend and backend allows developers to work independently and speeds up development.

Another important advantage is stateless communication. Every request sent by the client contains all required information, which improves scalability and server performance because the server does not need to maintain session data.

REST APIs also support standard HTTP methods such as GET, POST, PUT, PATCH, and DELETE, making CRUD operations simple and standardized. Additionally, REST architecture supports caching, which improves application speed and reduces server load.

Because of these advantages, REST APIs are widely used in modern applications that require smooth communication between frontend interfaces and backend services.

2. Explain how serializers act as a validation layer and how to implement custom field-level validation.

In Django REST Framework, serializers are responsible for converting complex model data into JSON format and converting incoming JSON data back into Python objects. Apart from data conversion, serializers also act as a validation layer that ensures only valid data is stored in the database.

Serializers automatically validate field types, required fields, maximum lengths, unique constraints, and other model-level validations. Before saving data, the serializer checks whether the incoming request follows the defined rules. If

validation fails, an error response is returned instead of saving incorrect data.

Custom field-level validation can be implemented using methods with the naming pattern `validate_<fieldname>`. These methods allow developers to define additional business rules for specific fields.

Example:

```
class DoctorSerializer(serializers.ModelSerializer):
```

```
    class Meta:
        model = Doctor
        fields = '__all__'

    def validate_name(self, value):
        if len(value) < 3:
            raise serializers.ValidationError("Doctor name must contain at least 3
characters.")
        return value
```

In this example, the `validate_name` method checks whether the doctor's name contains at least three characters. If the condition fails, a validation error is raised.

This validation layer improves data accuracy, application security, and database consistency.

3. Explain the importance of using appropriate HTTP status codes (201 Created vs 200 OK) for API outcomes.

HTTP status codes are important because they inform the client about the result of an API request. Proper status codes improve communication between the frontend and backend and help developers understand whether an operation was successful or failed.

The 200 OK status code indicates that the request was processed successfully and is generally used for successful GET, PUT, or PATCH requests where existing data is retrieved or updated.

The 201 Created status code is specifically used when a new resource is successfully created in the database through a POST request. It confirms that the server has successfully created a new record.

For example:

- GET request for doctor data → 200 OK
- POST request to create a new doctor → 201 Created

Using correct status codes improves API standards, debugging, frontend handling, and overall application reliability. Frontend applications often rely on these status codes to display appropriate success or error messages to users.

Incorrect status codes can create confusion and make error handling difficult. Therefore, developers should always use suitable HTTP status codes based on the API operation.

4. Explain why pagination is required when listing doctors and how it impacts database performance.

Pagination is required when handling large amounts of data because returning all records at once can slow down the API and increase server load. In a Doctor Finder application, the number of doctor records may grow significantly over time. Without pagination, every request would retrieve all records from the database, resulting in high memory usage and slower response times.

Pagination divides records into smaller pages and returns only a limited number of records per request. This improves API performance, reduces bandwidth usage, and provides faster responses to users.

For example:

- Page 1 → First 10 doctors
- Page 2 → Next 10 doctors

Django REST Framework provides built-in pagination classes such as:

- PageNumberPagination
- LimitOffsetPagination
- CursorPagination

Pagination improves database performance because the database processes fewer rows at a time. This reduces query execution time and server resource consumption.

It also improves user experience because frontend applications can load data gradually instead of waiting for thousands of records to load at once. Therefore, pagination is an important technique for building scalable and efficient APIs.

5. Explain the benefits of ViewSets over APIView for rapid CRUD development.

ViewSet in Django REST Framework simplify CRUD API development by combining multiple related operations into a single class. Instead of writing separate methods for GET, POST, PUT, and DELETE requests manually, ViewSets provide built-in actions for handling these operations automatically.

APIView gives developers more control but requires writing more code for each HTTP method. In contrast, ModelViewSet provides ready-made CRUD functionality such as:

- `list()`
- `retrieve()`
- `create()`
- `update()`
- `destroy()`

This significantly reduces boilerplate code and speeds up development.

Example:

```
class DoctorViewSet(viewsets.ModelViewSet):  
    queryset = Doctor.objects.all()  
    serializer_class = DoctorSerializer
```

By using routers, endpoints are automatically generated without manually defining URL patterns.

Benefits of ViewSets:

- Faster development
- Less repetitive code
- Cleaner project structure
- Easy integration with routers
- Built-in CRUD operations
- Better maintainability

ViewSet are highly useful in production APIs where rapid development and code consistency are important.

6. Explain the use of Atomic Transactions (transaction.atomic) when creating related doctor records to ensure data integrity.

Atomic Transactions ensure that database operations are completed fully or not completed at all. In Django, `transaction.atomic()` is used to maintain data integrity during critical operations involving multiple database actions.

When creating related doctor records, there may be multiple operations involved such as:

- Creating doctor details
- Creating specialization records
- Saving hospital information
- Updating related tables

If one operation succeeds and another fails, the database may end up with incomplete or corrupted data. Using `transaction.atomic()` prevents this issue by treating all operations as a single transaction.

Example:

```
from django.db import transaction
```

```
@transaction.atomic
def perform_create(self, serializer):
    serializer.save()
```

If any error occurs during the transaction, all changes are rolled back automatically. This ensures the database remains consistent and no partial data is stored.

Benefits of Atomic Transactions:

- Prevents partial database updates
- Maintains data consistency
- Ensures reliable operations
- Supports rollback during failures
- Improves application stability

Atomic transactions are especially important in production systems where multiple related records must be saved together safely.